

Universidad Nacional de Loja

FACULTAD DE LA ENERGÍA, LAS INDUSTRIAS Y LOS RECURSOS NATURALES NO RENOVABLES

CARRERA DE INGENIERÍA EN CIENCIAS DE LA COMPUTACIÓN

TERCER CICLO "A"

ASIGNATURA: *Base De Datos*

ELABORADO POR:

Lucas Sebastian Orellana Martinez

Docente: *Ing. Renne Guaman*

Unidad: 1era Unidad

LOJA – ECUADOR

1. TITULO

Sistema de Gestión de Eventos y Venta de Entradas Digitales con Memorias de Eventos Anteriores

2. CONTEXTO DEL PROBLEMA

Actualmente, las productoras y organizadores de eventos necesitan herramientas que les permitan administrar sus eventos, controlar la venta de entradas y registrar la información de sus compradores. Sin una base de datos organizada, pueden presentarse dificultades para controlar las entradas disponibles, consultar las ventas y conocer la cantidad de asistentes. Por esta razón, se propone desarrollar un Sistema de Gestión de Eventos y Venta de Entradas Digitales, donde diferentes organizadores puedan publicar sus eventos, establecer precios y preventas, registrar compras y generar entradas con códigos QR individuales. Como función adicional, el sistema contará con un apartado de Memorias, donde se podrán consultar eventos anteriores, conocer los artistas que participaron y visualizar o descargar fotografías publicadas por los organizadores.

Para la Unidad 1, el proyecto se enfocará en el diseño e implementación de la base de datos en MySQL, utilizando modelos Entidad-Relación, tablas relacionadas y consultas SQL para comprobar su funcionamiento.

3. OBJETIVOS

3.1. Objetivo General

Diseñar e implementar una base de datos relacional para un sistema de gestión de eventos y venta de entradas digitales de múltiples organizadores, utilizando el modelo Entidad-Relación, el modelo relacional y sentencias SQL, para administrar eventos, tipos de entradas, compras y tickets, e incorporar el historial de eventos con artistas y fotografías como función complementaria.

3.2. Objetivos Específicos

- Elaborar el modelo Entidad-Relación y el modelo relacional del sistema, identificando las entidades, atributos, claves y cardinalidades necesarias para organizar eventos, entradas digitales, compras y memorias de eventos anteriores.

- Implementar las tablas de la base de datos mediante sentencias DDL y DML y desarrollar cinco consultas de algebra relacional traducidas a SQL que permitan consultar ventas, entradas, organizadores y memorias

4. MODELOS CONCEPTUALES DE BASES DE DATOS

4.1. Modelo Entidad-Relación (E-R)

El modelo contiene nueve entidades. Las principales operaciones de venta se representan mediante:

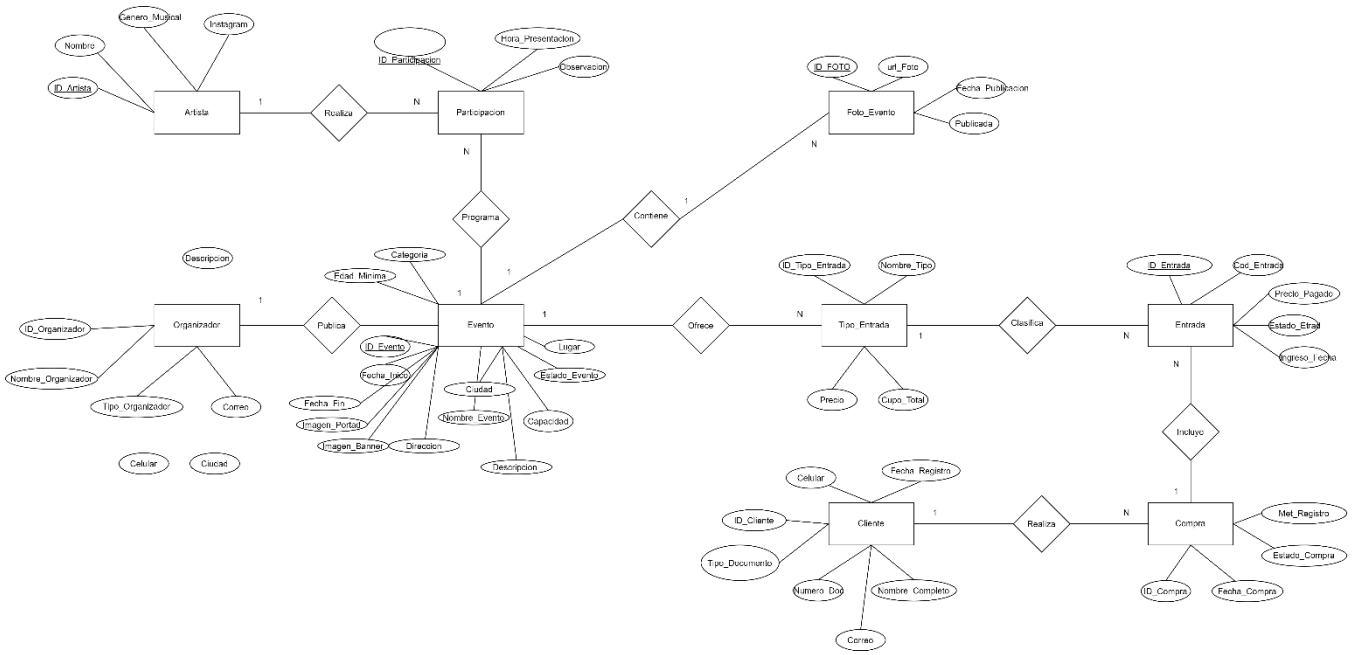
ORGANIZADOR, EVENTO, TIPO_ENTRADA, CLIENTE, COMPRA y ENTRADA.

El complemento de memorias se apoya en:

ARTISTA, PARTICIPACION y FOTO_EVENTO. PARTICIPACION

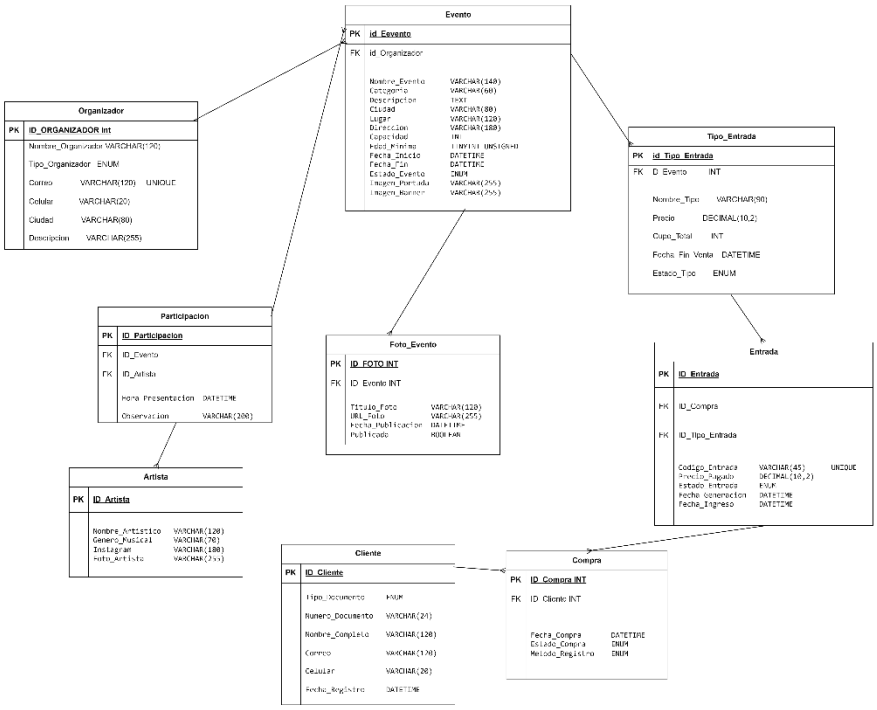
LINK: https://drive.google.com/file/d/1sJBCLi6HMAOiFtYepOWm_eGQT7LQ0lk0/view?usp=sharing

Relación	**Cardinalidad**
ORGANIZADOR — Publica — EVENTO	1:N
EVENTO — Ofrece — TIPO_ENTRADA	1:N
CLIENTE — Realiza — COMPRA	1:N
COMPRA — Incluye — ENTRADA	1:N
TIPO_ENTRADA — Clasifica — ENTRADA	1:N
EVENTO — Contiene — FOTO_EVENTO	1:N
EVENTO — Programa — PARTICIPACION	1:N
ARTISTA — Realiza — PARTICIPACION	1:N



4.2. Modelo Relacional

LINK:
https://drive.google.com/file/d/1sJBCLi6HMAOiFtYepOWm_eGQT7LQ0lk0/view?usp=sharing



Notación: PK = clave primaria; FK = clave foránea; UQ = restricción de unicidad.

ORGANIZADOR:

ID_Organizador INT [PK];

Nombre_Organizador VARCHAR(120);

Tipo_Organizador ENUM;

Correo VARCHAR(120) [UQ];

Celular VARCHAR(20);

Ciudad VARCHAR(80);

Descripcion VARCHAR(255).

EVENTO:

ID_Evento INT [PK];

ID_Organizador INT [FK → ORGANIZADOR];

Nombre_Evento VARCHAR(140);

Categoria VARCHAR(60);

Descripcion TEXT;

Ciudad VARCHAR(80);

Lugar VARCHAR(120);

Direccion VARCHAR(180);

Capacidad INT; Edad_Minima TINYINT;

Fecha_Inicio y Fecha_Fin DATETIME;

Estado_Evento ENUM;

Imagen_Portada y Imagen_Banner VARCHAR(255).

TIPO_ENTRADA:

ID_Tipo_Entrada INT [PK];

ID_Evento INT [FK → EVENTO];

Nombre_Tipo VARCHAR(90);

Precio DECIMAL(10,2);

Cupo_Total INT;

Fecha_Fin_Venta DATETIME;

Estado_Tipo ENUM.

CLIENTE:

ID_Cliente INT [PK];

Tipo_Documento ENUM;

Numero_Documento VARCHAR(24);

Nombre_Completo VARCHAR(120);

Correo VARCHAR(120);

Celular VARCHAR(20);

Fecha_Registro DATETIME. UQ(Tipo_Documento, Numero_Documento).

COMPRA:

ID_Compra INT [PK];

ID_Cliente INT [FK → CLIENTE];

Fecha_Compra DATETIME;

Estado_Compra ENUM;

Metodo_Registro ENUM.

ENTRADA:

ID_Entrada INT [PK];

ID_Compra INT [FK → COMPRA];

ID_Tipo_Entrada INT [FK → TIPO_ENTRADA];

Codigo_Entrada VARCHAR(45) [UQ];

Precio_Pagado DECIMAL(10,2);

Estado_Entrada ENUM;

Fecha_Ingreso DATETIME NULL;

ARTISTA:

ID_Artista INT [PK];

Nombre_Artistico VARCHAR(120);

Genero_Musical VARCHAR(70);

Instagram VARCHAR(180);

Foto_Artista VARCHAR(255).

PARTICIPACION:

ID_Participacion INT [PK];

ID_Evento INT [FK → EVENTO];

ID_Artista INT [FK → ARTISTA];

Hora_Presentacion DATETIME;

Observacion VARCHAR(200);

UQ(ID_Evento, ID_Artista).

FOTO_EVENTO:

ID_Foto INT [PK];

ID_Evento INT [FK → EVENTO];

Titulo_Foto VARCHAR(120);

URL_Foto VARCHAR(255) [UQ];

Fecha_Publicacion DATETIME;

Publicada BOOLEAN.

CÓDIGO SQL

5.1. Creación de Tablas (DDL)

En esta sección se utiliza el lenguaje DDL para crear las nueve tablas de nuestra base de datos en MySQL. Se establecen las claves primarias, claves foráneas y restricciones necesarias para relacionar correctamente la información de los organizadores, eventos, clientes, compras, entradas, artistas y fotografías.

```
-Tabla  ORGANIZADOR
CREATE TABLE ORGANIZADOR (
ID_Organizador INT AUTO_INCREMENT PRIMARY KEY,
Nombre_Organizador VARCHAR(120) NOT NULL,
Tipo_Organizador ENUM('Productora','Artista','Empresa','Particular') NOT NULL,
Correo VARCHAR(120) NOT NULL UNIQUE,
Celular VARCHAR(20),
Ciudad VARCHAR(80) NOT NULL,
Descripcion VARCHAR(255)
) ENGINE=InnoDB;
Tabla. EVENTO
CREATE TABLE EVENTO (
ID_Evento INT AUTO_INCREMENT PRIMARY KEY,
ID_Organizador INT NOT NULL,
Nombre_Evento VARCHAR(140) NOT NULL,
Categoria VARCHAR(60) NOT NULL,
Descripcion TEXT,
```

```
Ciudad VARCHAR(80) NOT NULL,
Lugar VARCHAR(120) NOT NULL,
Direccion VARCHAR(180),
Capacidad INT NOT NULL,
Edad_Minima TINYINT UNSIGNED NOT NULL DEFAULT 0,
Fecha_Inicio DATETIME NOT NULL,
Fecha_Fin DATETIME NOT NULL,
Estado_Evento ENUM('Borrador','Publicado','Cancelado',
'Finalizado') NOT NULL DEFAULT 'Borrador',
Imagen_Portada VARCHAR(255),
Imagen_Banner VARCHAR(255),
CONSTRAINT FK_Evento_Organizador FOREIGN KEY (ID_Organizador)
REFERENCES ORGANIZADOR(ID_Organizador) ON UPDATE CASCADE ON DELETE RESTRICT,
CONSTRAINT CHK_Evento_Capacidad CHECK (Capacidad > 0),
CONSTRAINT CHK_Evento_Fechas CHECK (Fecha_Fin > Fecha_Inicio),
CONSTRAINT CHK_Evento_Edad CHECK (Edad_Minima <= 99)
) ENGINE=InnoDB;
```

Tabla. TIPO_ENTRADA

```
CREATE TABLE TIPO_ENTRADA (
ID_Tipo_Entrada INT AUTO_INCREMENT PRIMARY KEY,
ID_Evento INT NOT NULL,
Nombre_Tipo VARCHAR(90) NOT NULL,
Precio DECIMAL(10,2) NOT NULL,
Cupo_Total INT NOT NULL,
Fecha_Fin_Venta DATETIME,
Estado_Tipo ENUM('Disponible','Cerrado') NOT NULL DEFAULT 'Disponible',
CONSTRAINT FK_Tipo_Entrada_Evento FOREIGN KEY (ID_Evento)
REFERENCES EVENTO(ID_Evento) ON UPDATE CASCADE ON DELETE RESTRICT,
CONSTRAINT UQ_Tipo_Entrada_Nombre UNIQUE (ID_Evento, Nombre_Tipo),
CONSTRAINT CHK_Tipo_Entrada_Precio CHECK (Precio >= 0),
CONSTRAINT CHK_Tipo_Entrada_Cupo CHECK (Cupo_Total > 0)
) ENGINE=InnoDB;
```

Tabla CLIENTE

```
CREATE TABLE CLIENTE (
ID_Cliente INT AUTO_INCREMENT PRIMARY KEY,
Tipo_Documento ENUM('Cedula','Pasaporte') NOT NULL,
Numero_Documento VARCHAR(24) NOT NULL,
Nombre_Completo VARCHAR(120) NOT NULL,
Correo VARCHAR(120) NOT NULL,
Celular VARCHAR(20),
Fecha_Registro DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT UQ_Cliente_Documento UNIQUE (Tipo_Documento, Numero_Documento)
) ENGINE=InnoDB;
```

Tabla COMPRA

```
CREATE TABLE COMPRA (
ID_Compra INT AUTO_INCREMENT PRIMARY KEY,
ID_Cliente INT NOT NULL,
Fecha_Compra DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
Estado_Compra ENUM('Pendiente','Confirmada','Cancelada') NOT NULL DEFAULT 'Pendiente',
Metodo_Registro ENUM('Web','Manual') NOT NULL DEFAULT 'Web',
CONSTRAINT FK_Compra_Cliente FOREIGN KEY (ID_Cliente)
REFERENCES CLIENTE(ID_Cliente) ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB;
```

Tabla ENTRADA

```
CREATE TABLE ENTRADA (
ID_Entrada INT AUTO_INCREMENT PRIMARY KEY,
ID_Compra INT NOT NULL,
ID_Tipo_Entrada INT NOT NULL,
Codigo_Entrada VARCHAR(45) NOT NULL UNIQUE,
Precio_Pagado DECIMAL(10,2) NOT NULL,
Estado_Entrada ENUM('Emitida','Utilizada','Anulada') NOT NULL DEFAULT 'Emitida',
Fecha_Generacion DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
Fecha_Ingreso DATETIME NULL,
CONSTRAINT FK_Entrada_Compra FOREIGN KEY (ID_Compra)
REFERENCES COMPRA(ID_Compra) ON UPDATE CASCADE ON DELETE RESTRICT,
CONSTRAINT FK_Entrada_Tipo FOREIGN KEY (ID_Tipo_Entrada)
REFERENCES TIPO_ENTRADA(ID_Tipo_Entrada) ON UPDATE CASCADE ON DELETE RESTRICT,
CONSTRAINT CHK_Entrada_Precio CHECK (Precio_Pagado >= 0)
) ENGINE=InnoDB;
```

Tabla. ARTISTA

```
CREATE TABLE ARTISTA (
ID_Artista INT AUTO_INCREMENT PRIMARY KEY,
Nombre_Artistico VARCHAR(120) NOT NULL,
Genero_Musical VARCHAR(70),
Instagram VARCHAR(180),
Foto_Artista VARCHAR(255)
) ENGINE=InnoDB;
```

Tabla PARTICIPACION

```
CREATE TABLE PARTICIPACION (
ID_Participacion INT AUTO_INCREMENT PRIMARY KEY,
ID_Evento INT NOT NULL,
ID_Artista INT NOT NULL,
Hora_Presentacion DATETIME,
Observacion VARCHAR(200),
CONSTRAINT FK_Participacion_Evento FOREIGN KEY (ID_Evento)
REFERENCES EVENTO(ID_Evento) ON UPDATE CASCADE ON DELETE RESTRICT,
```

```
CONSTRAINT FK_Participacion_Artista FOREIGN KEY (ID_Artista)
REFERENCES ARTISTA(ID_Artista) ON UPDATE CASCADE ON DELETE RESTRICT,
CONSTRAINT UQ_Participacion UNIQUE (ID_Evento, ID_Artista)
) ENGINE=InnoDB;
```

Tabla FOTO_EVENTO

```
CREATE TABLE FOTO_EVENTO (
ID_Foto INT AUTO_INCREMENT PRIMARY KEY,
ID_Evento INT NOT NULL,
Titulo_Foto VARCHAR(120),
URL_Foto VARCHAR(255) NOT NULL,
Fecha_Publicacion DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
Publicada BOOLEAN NOT NULL DEFAULT FALSE,
CONSTRAINT FK_Foto_Evento FOREIGN KEY (ID_Evento)
REFERENCES EVENTO(ID_Evento) ON UPDATE CASCADE ON DELETE RESTRICT,
CONSTRAINT UQ_Foto_Url UNIQUE (URL_Foto)
) ENGINE=InnoDB;
```

5.2. Inserción de Datos (DML)

En esta sección se utiliza el lenguaje DML para ingresar datos ficticios en las nueve tablas de nuestra base de datos. Se registraron cinco organizadores, cinco eventos, ocho tipos de entradas, cinco clientes, siete compras, doce entradas, cinco artistas, siete participaciones y cinco fotografías. Estos datos nos servirán para realizar las consultas SQL y comprobar las relaciones entre las tablas.

--Registros de ORGANIZADOR

```
INSERT INTO ORGANIZADOR
(Nombre_Organizador, Tipo_Organizador, Correo, Celular, Ciudad, Descripcion) VALUES
('RESAK Premium','Productora','resak.ejemplo@example.com','0990000001','Loja',
'Eventos musicales'),
('Escena Andina','Productora','andina@example.com','0990000002','Cuenca',
'Encuentros culturales'),
('Ruta Sonora','Empresa','ruta@example.com','0990000003','Quito','Festivales y conciertos'),
('Luna Urbana','Productora','luna@example.com','0990000004','Loja','Fiestas tematicas'),
('Horizonte Artistico','Artista','horizonte@example.com','0990000005','Ambato',
'Presentaciones en vivo');
```

Registros de EVENTO

```
INSERT INTO EVENTO
(ID_Organizador, Nombre_Evento, Categoria, Descripcion, Ciudad, Lugar, Direccion, Capacidad,
Edad_Minima, Fecha_Inicio, Fecha_Fin, Estado_Evento, Imagen_Portada, Imagen_Banner) VALUES
(1,'Noche RESAK','Fiesta','Evento demostrativo de RESAK','Loja','Centro de eventos A',
'Zona centro',350,18,'2026-12-12 20:00:00','2026-12-13 03:00:00','Publicado',
'/media/resak_portada.jpg','/media/resak_banner.jpg'),
(3,'Festival Horizonte','Festival','Festival musical de prueba','Quito','Recinto B',
'Av. Principal',600,0,'2026-12-19 17:00:00','2026-12-20 01:00:00','Publicado',
'/media/festival_portada.jpg','/media/festival_banner.jpg'),
(4,'Beats de Ciudad','Concierto','Sesion electronica','Loja','Sala C','Calle Central',250,18,
'2027-01-09 21:00:00','2027-01-10 02:00:00','Publicado','/media/beats_portada.jpg',NULL),
```

(2,'Encuentro Cultural','Cultural','Encuentro finalizado','Cuenca','Teatro D',
'Sector Historico',180,0,'2026-08-20 18:00:00','2026-08-20 23:00:00','Finalizado',
'/media/cultural_portada.jpg',NULL),
(5,'Sesion Acustica','Concierto','Evento en preparacion','Ambato','Auditorio E',
'Av. del Parque',150,0,'2027-02-06 19:00:00','2027-02-06 23:00:00','Borrador',NULL,NULL);

Registros de TIPO_ENTRADA

INSERT INTO TIPO_ENTRADA
(ID_Evento, Nombre_Tipo, Precio, Cupo_Total, Fecha_Fin_Venta, Estado_Tipo) VALUES
(1,'General Preventa 1',8.00,100,'2026-12-01 23:59:00','Disponible'),
(1,'General Preventa 2',12.00,100,'2026-12-11 23:59:00','Disponible'),
(1,'VIP',15.00,50,'2026-12-12 21:00:00','Disponible'),
(2,'General',12.00,200,'2026-12-19 19:00:00','Disponible'),
(2,'VIP',20.00,40,'2026-12-19 19:00:00','Disponible'),
(3,'General',10.00,150,'2027-01-09 20:00:00','Disponible'),
(4,'General',6.00,120,'2026-08-20 17:00:00','Cerrado'),
(5,'General',5.00,100,'2027-02-06 18:00:00','Cerrado');

Registros de CLIENTE

INSERT INTO CLIENTE
(Tipo_Documento, Numero_Documento, Nombre_Completo, Correo, Celular) VALUES
('Cedula','0000000001','Cliente de Prueba 1','cliente1@example.com','0991111001'),
('Cedula','0000000002','Cliente de Prueba 2','cliente2@example.com','0991111002'),
('Pasaporte','PAS-EJ-003','Cliente de Prueba 3','cliente3@example.com','0991111003'),
('Cedula','0000000004','Cliente de Prueba 4','cliente4@example.com','0991111004'),
('Pasaporte','PAS-EJ-005','Cliente de Prueba 5','cliente5@example.com','0991111005');

Registros de COMPRA

INSERT INTO COMPRA
(ID_Cliente, Fecha_Compra, Estado_Compra, Metodo_Registro) VALUES
(1,'2026-10-10 10:00:00','Confirmada','Web'),
(2,'2026-10-11 11:30:00','Confirmada','Web'),
(3,'2026-10-12 16:00:00','Confirmada','Manual'),
(4,'2026-08-19 18:00:00','Confirmada','Web'),
(5,'2026-10-13 09:00:00','Confirmada','Web'),
(1,'2026-10-14 13:00:00','Confirmada','Manual'),
(2,'2026-10-15 08:00:00','Cancelada','Web');

Registros de ENTRADA

INSERT INTO ENTRADA
(ID_Compra, ID_Tipo_Entrada, Codigo_Entrada, Precio_Pagado, Estado_Entrada,
Fecha_Generacion) VALUES
(1,1,'EJ-0001',8.00,'Emitida','2026-10-10 10:00:00'),
(1,1,'EJ-0002',8.00,'Emitida','2026-10-10 10:00:00'),
(2,3,'EJ-0003',15.00,'Emitida','2026-10-11 11:30:00'),
(3,4,'EJ-0004',12.00,'Emitida','2026-10-12 16:00:00'),
(3,4,'EJ-0005',12.00,'Emitida','2026-10-12 16:00:00'),

+

(4,7,'EJ-0006',6.00,'Utilizada','2026-08-19 18:00:00'),

(5,6,'EJ-0007',10.00,'Emitida','2026-10-13 09:00:00'),

(6,5,'EJ-0008',20.00,'Emitida','2026-10-14 13:00:00'),

(6,5,'EJ-0009',20.00,'Emitida','2026-10-14 13:00:00'),

(2,1,'EJ-0010',8.00,'Emitida','2026-10-11 11:30:00'),

(4,7,'EJ-0011',6.00,'Utilizada','2026-08-19 18:00:00'),

(7,8,'EJ-0012',5.00,'Anulada','2026-10-15 08:00:00');

Registros de ARTISTA

INSERT INTO ARTISTA

(Nombre_Artistico, Genero_Musical, Instagram, Foto_Artista) VALUES

('DJ Aurora','House','https://instagram.com/ejemplo_aurora','/media/artistas/aurora.jpg'),

('DJ Nebula','Tech House','https://instagram.com/ejemplo_nebula',

'/media/artistas/nebula.jpg'),

('Banda Ruta','Pop','https://instagram.com/ejemplo_bandaruta','/media/artistas/ruta.jpg'),

('Solista Luz','Pop','https://instagram.com/ejemplo_solista','/media/artistas/luz.jpg'),

('Orquesta Horizonte','Tropical','https://instagram.com/ejemplo_orquesta',

'/media/artistas/horizonte.jpg');

Registros de PARTICIPACION

INSERT INTO PARTICIPACION

(ID_Evento, ID_Artista, Hora_Presentacion, Observacion) VALUES

(1,1,'2026-12-12 22:00:00','Apertura'),

(1,2,'2026-12-13 00:00:00','Cierre'),

(2,3,'2026-12-19 20:00:00','Concierto'),

(2,4,'2026-12-19 21:00:00','Invitada'),

(3,1,'2027-01-09 23:00:00','Set principal'),

(4,5,'2026-08-20 19:00:00','Presentacion central'),

(5,4,'2027-02-06 20:00:00','Programacion preliminar');

Registros de FOTO_EVENTO

INSERT INTO FOTO_EVENTO

(ID_Evento, Titulo_Foto, URL_Foto, Fecha_Publicacion, Publicada) VALUES

(4,'Apertura','/media/memorias/cultural_01.jpg','2026-08-21 10:00:00',TRUE),

(4,'Presentacion','/media/memorias/cultural_02.jpg','2026-08-21 10:05:00',TRUE),

(4,'Publico','/media/memorias/cultural_03.jpg','2026-08-21 10:10:00',TRUE),

(4,'Escenario','/media/memorias/cultural_04.jpg','2026-08-21 10:15:00',TRUE),

(4,'Cierre','/media/memorias/cultural_05.jpg','2026-08-21 10:20:00',TRUE);

Actualización de entradas utilizadas

UPDATE ENTRADA SET Fecha_Ingreso = '2026-08-20 19:05:00' WHERE Codigo_Entrada = 'EJ-0006';

UPDATE ENTRADA SET Fecha_Ingreso = '2026-08-20 19:12:00' WHERE Codigo_Entrada = 'EJ-0011';

CONSULTA DE ÁLGEBRA RELACIONAL Y TRADUCIR A SQL

Presentar 5 consultas formuladas primero en Álgebra Relacional y luego traducidas al lenguaje SQL. Las consultas deben tener diferente nivel de complejidad (relación de dos tablas o más) e involucrar distintos operadores del álgebra relacional.

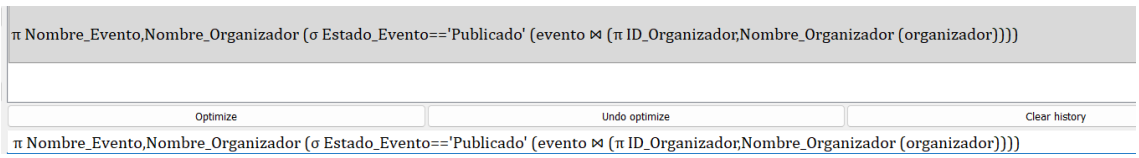
Operadores del algebra relacional a utilizar: *Seleccion (sigma), Proyeccion (pi), Union, Interseccion, Diferencia, Producto cartesiano, Reunion natural (JOIN), Division. Cada consulta debe usar al menos un operador diferente.*

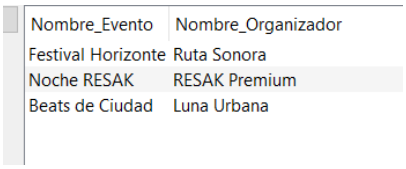
Las consultas propuestas son sólo ejemplos. El estudiante debe presentar consultas complejas

Consulta 1 — (operador: Selección σ.. etc)

Mostrar los eventos que se encuentran publicados en la plataforma, junto con el nombre de sus organizadores, utilizando las tablas EVENTO y ORGANIZADOR.

Álgebra Relacional:





Nombre_Evento	Nombre_Organizador
Festival Horizonte	Ruta Sonora
Noche RESAK	RESAK Premium
Beats de Ciudad	Luna Urbana



SQL:

```
SELECT
   EVENTO.Nombre_Evento,
   ORGANIZADOR.Nombre_Organizador
FROM EVENTO
INNER JOIN ORGANIZADOR
   ON EVENTO.ID_Organizador =
      ORGANIZADOR.ID_Organizador
WHERE EVENTO.Estado_Evento = 'Publicado';
```

Se obtuvieron tres eventos publicados: Noche RESAK, Festival Horizonte y Beats de Ciudad, cada uno con su respectivo organizador. Esto nos permite comprobar que las tablas están relacionadas correctamente y que podemos consultar únicamente los eventos publicados.

.

Consulta 2 — Proyección (operador: Proyección π)

Mostrar los eventos que ya finalizaron, los artistas que participaron, sus cuentas de Instagram y las fotografías que están publicadas en la plataforma.

Álgebra Relacional:

π Nombre_Evento,Nombre_Organizador (σ Estado_Evento=='Publicado' (evento ⋈ (π ID_Organizador,Nombre_Organizador (organizador))))

π Nombre_Evento,Nombre_Artistico,Instagram,URL_Foto (σ Estado_Evento=='Finalizado' and Publicada==1 (evento ⋈ foto_evento ⋈ participacion ⋈ artista))

ons Settings Help

Nombre_Evento	Nombre_Artistico	Instagram	URL_Foto
Encuentro Cultural	Orquesta Horizonte	https://instagram.com/ejemplo_orquesta	/media/memorias/cultural_03.jpg
Encuentro Cultural	Orquesta Horizonte	https://instagram.com/ejemplo_orquesta	/media/memorias/cultural_01.jpg
Encuentro Cultural	Orquesta Horizonte	https://instagram.com/ejemplo_orquesta	/media/memorias/cultural_04.jpg
Encuentro Cultural	Orquesta Horizonte	https://instagram.com/ejemplo_orquesta	/media/memorias/cultural_02.jpg
Encuentro Cultural	Orquesta Horizonte	https://instagram.com/ejemplo_orquesta	/media/memorias/cultural_05.jpg

Mostrar todo | Número de filas: 25 | Filtrar filas: Buscar en esta tabla | Ordenar según la clave:

Opciones extra

Nombre_Evento	Nombre_Artistico	Instagram	URL_Foto
Encuentro Cultural	Orquesta Horizonte	https://instagram.com/ejemplo_orquesta	/media/memorias/cultural_01.jpg
Encuentro Cultural	Orquesta Horizonte	https://instagram.com/ejemplo_orquesta	/media/memorias/cultural_02.jpg
Encuentro Cultural	Orquesta Horizonte	https://instagram.com/ejemplo_orquesta	/media/memorias/cultural_03.jpg
Encuentro Cultural	Orquesta Horizonte	https://instagram.com/ejemplo_orquesta	/media/memorias/cultural_04.jpg
Encuentro Cultural	Orquesta Horizonte	https://instagram.com/ejemplo_orquesta	/media/memorias/cultural_05.jpg

SQL:

```
SELECT DISTINCT
    EVENTO.Nombre_Evento,
    ARTISTA.Nombre_Artistico,
    ARTISTA.Instagram,
    FOTO_EVENTO.URL_Foto
FROM EVENTO
INNER JOIN FOTO_EVENTO
    ON EVENTO.ID_Evento = FOTO_EVENTO.ID_Evento
INNER JOIN PARTICIPACION
    ON EVENTO.ID_Evento = PARTICIPACION.ID_Evento
INNER JOIN ARTISTA
    ON PARTICIPACION.ID_Artista = ARTISTA.ID_Artista
WHERE EVENTO.Estado_Evento = 'Finalizado'
    AND FOTO_EVENTO.Publicada = TRUE;
```

Se obtendrán cinco fotografías publicadas del evento Encuentro Cultural, junto con la información del artista Orquesta Horizonte. Esto permite comprobar que podemos consultar las memorias de los eventos anteriores relacionando las tablas correspondientes.

Consulta 3 — Selección con Proyección (σ + π combinados)

Mostrar los tipos de entradas que tienen un precio de $12 o menos y que pertenecen a eventos publicados.

Álgebra Relacional:

```
π Nombre_Evento,Nombre_Tipo,Precio (σ Estado_Evento=='Publicado' and Precio<=12 (evento ⋈ tipo_entrada))
```

Nombre_Evento	Nombre_Tipo	Precio
Noche RESAK	General Preventa 2	12.0
Noche RESAK	General Preventa 1	8.0
Beats de Ciudad	General	10.0
Festival Horizonte	General	12.0

Nombre_Evento	Nombre_Tipo	Precio
Noche RESAK	General Preventa 1	8.00
Noche RESAK	General Preventa 2	12.00
Festival Horizonte	General	12.00
Beats de Ciudad	General	10.00

SQL:

```
SELECT
   EVENTO.Nombre_Evento,
   TIPO_ENTRADA.Nombre_Tipo,
   TIPO_ENTRADA.Precio
FROM EVENTO
INNER JOIN TIPO_ENTRADA
   ON EVENTO.ID_Evento = TIPO_ENTRADA.ID_Evento
WHERE EVENTO.Estado_Evento = 'Publicado'
   AND TIPO_ENTRADA.Precio <= 12.00;
```

Se obtendrán cuatro tipos de entradas que cumplen con el precio establecido, pertenecientes a Noche RESAK, Festival Horizonte y Beats de Ciudad. Esto permite consultar las diferentes entradas según su precio y el estado del evento.

Consulta 4 — Reunión natural / JOIN (operador: ⋈)

Mostrar las entradas adquiridas por los clientes, junto con su nombre, código de entrada, evento y tipo de entrada, tomando en cuenta únicamente las compras confirmadas

Álgebra Relacional:

$$\pi\ \text{Nombre_Completo,Codigo_Entrada,Nombre_Evento,Nombre_Tipo}\ (\sigma\ \text{Estado_Compra=='Confirmada' and Estado_Entrada!='Anulada'}\ (\text{cliente} \bowtie \text{compra} \bowtie \text{entrada} \bowtie$$

Nombre_Completo	Codigo_Entrada	Nombre_Evento	Nombre_Tipo
Cliente de Prueba 4	EJ-0006	Encuentro Cultural	General
Cliente de Prueba 5	EJ-0007	Beats de Ciudad	General
Cliente de Prueba 3	EJ-0005	Festival Horizonte	General
Cliente de Prueba 2	EJ-0003	Noche RESAK	VIP
Cliente de Prueba 1	EJ-0001	Noche RESAK	General Preventa 1
Cliente de Prueba 4	EJ-0011	Encuentro Cultural	General
Cliente de Prueba 2	EJ-0010	Noche RESAK	General Preventa 1
Cliente de Prueba 1	EJ-0008	Festival Horizonte	VIP
Cliente de Prueba 1	EJ-0002	Noche RESAK	General Preventa 1
Cliente de Prueba 1	EJ-0009	Festival Horizonte	VIP
Cliente de Prueba 3	EJ-0004	Festival Horizonte	General

Nombre_Completo	Codigo_Entrada	Nombre_Evento	Nombre_Tipo
Cliente de Prueba 1	EJ-0001	Noche RESAK	General Preventa 1
Cliente de Prueba 1	EJ-0002	Noche RESAK	General Preventa 1
Cliente de Prueba 2	EJ-0010	Noche RESAK	General Preventa 1
Cliente de Prueba 2	EJ-0003	Noche RESAK	VIP
Cliente de Prueba 3	EJ-0004	Festival Horizonte	General
Cliente de Prueba 3	EJ-0005	Festival Horizonte	General
Cliente de Prueba 1	EJ-0008	Festival Horizonte	VIP
Cliente de Prueba 1	EJ-0009	Festival Horizonte	VIP
Cliente de Prueba 5	EJ-0007	Beats de Ciudad	General
Cliente de Prueba 4	EJ-0006	Encuentro Cultural	General
Cliente de Prueba 4	EJ-0011	Encuentro Cultural	General

SQL:

```
SELECT
   CLIENTE.Nombre_Completo,
   ENTRADA.Codigo_Entrada,
   EVENTO.Nombre_Evento,
   TIPO_ENTRADA.Nombre_Tipo
FROM CLIENTE
INNER JOIN COMPRA
   ON CLIENTE.ID_Cliente = COMPRA.ID_Cliente
INNER JOIN ENTRADA
   ON COMPRA.ID_Compra = ENTRADA.ID_Compra
INNER JOIN TIPO_ENTRADA
   ON ENTRADA.ID_Tipo_Entrada =
      TIPO_ENTRADA.ID_Tipo_Entrada
INNER JOIN EVENTO
   ON TIPO_ENTRADA.ID_Evento = EVENTO.ID_Evento
WHERE COMPRA.Estado_Compra = 'Confirmada'
   AND ENTRADA.Estado_Entrada <> 'Anulada';
```

Se obtendrán 11 entradas pertenecientes a compras confirmadas, mostrando los datos de cada comprador y el evento correspondiente. Esto nos permite comprobar cómo se relacionan las tablas CLIENTE, COMPRA, ENTRADA, TIPO_ENTRADA y EVENTO.

Consulta 5 — Consulta con agrupamiento y función de agregación

Conocer cuántas entradas se vendieron en cada evento y cuánto dinero se recaudó, tomando en cuenta las compras confirmadas y las entradas que no están anuladas.

Álgebra Relacional:

$$\pi\ \text{ID_Evento,Nombre_Evento,ID_Entrada,Precio_Pagado}\ (\sigma\ \text{Estado_Compra=='Confirmada' and Estado_Entrada!='Anulada'}\ (\text{evento} \bowtie \text{tipo_entrada} \bowtie \text{entrada} \bowtie \text{compra}$$

ID_Evento	Nombre_Evento	ID_Entrada	Precio_Pagado
1	Noche RESAK	3	15.0
4	Encuentro Cultural	11	6.0
4	Encuentro Cultural	6	6.0
2	Festival Horizonte	9	20.0
1	Noche RESAK	10	8.0
1	Noche RESAK	1	8.0
3	Beats de Ciudad	7	10.0
1	Noche RESAK	2	8.0
2	Festival Horizonte	8	20.0
2	Festival Horizonte	4	12.0
2	Festival Horizonte	5	12.0

Nombre_Evento	Entradas_Vendidas	Ingresos_Simulados
Noche RESAK	4	39.00
Festival Horizonte	4	64.00
Beats de Ciudad	1	10.00
Encuentro Cultural	2	12.00

SQL:

```
SELECT
   EVENTO.Nombre_Evento,
   COUNT(ENTRADA.ID_Entrada) AS Entradas_Vendidas,
   SUM(ENTRADA.Precio_Pagado) AS Ingresos_Simulados
FROM EVENTO
INNER JOIN TIPO_ENTRADA
   ON EVENTO.ID_Evento = TIPO_ENTRADA.ID_Evento
INNER JOIN ENTRADA
   ON TIPO_ENTRADA.ID_Tipo_Entrada =
      ENTRADA.ID_Tipo_Entrada
INNER JOIN COMPRA
   ON ENTRADA.ID_Compra = COMPRA.ID_Compra
WHERE COMPRA.Estado_Compra = 'Confirmada'
   AND ENTRADA.Estado_Entrada <> 'Anulada'
GROUP BY
   EVENTO.ID_Evento,
   EVENTO.Nombre_Evento
ORDER BY EVENTO.ID_Evento;
;
```

Explicar qué resumen estadístico o agrupamiento se obtendrá.

Se obtendrán las ventas de cuatro eventos, con un total de 11 entradas vendidas y $125,00 de ingresos simulados. Esto nos permite conocer qué eventos tuvieron mayores ventas mediante las funciones COUNT(), SUM() y GROUP BY.

CONCLUSIONES

1. Al realizar el modelo Entidad-Relación, pude identificar las entidades necesarias para nuestro sistema de venta de entradas, como los organizadores, eventos, clientes, compras y entradas. También incluí los artistas y las fotografías para que se puedan consultar los eventos anteriores. Esto me permitió entender mejor cómo se relaciona la información dentro de una base de datos.
2. Al trabajar con DDL y DML, aprendí cómo crear las tablas, establecer las claves primarias y foráneas e ingresar información en la base de datos. También comprendí la importancia de relacionar correctamente las tablas para organizar las compras, los tipos de entradas y los tickets que se generan.
3. Con el álgebra relacional y las consultas SQL, pude comprender cómo obtener información de las diferentes tablas, como los eventos publicados, las entradas vendidas y los clientes que realizaron compras. Además, pude realizar consultas para conocer las ventas de cada evento y las fotografías publicadas. Esto me ayudó a entender cómo una base de datos puede facilitar la administración de un sistema de venta de entradas.

BIBLIOGRAFÍA

[1] Universidad Nacional de Loja, “Evaluación U1: Informe de fin de unidad,” guía de la asignatura Base de Datos, documento proporcionado por el docente, 2026.

[2] Meet2Go, “Plataforma de eventos y entradas,” https://www.meet2go.com/ (consulta: 8 de noviembre de 2026).

[3] FLAME, “Plataforma de eventos,” https://www.flme.co/ (consulta: 8 de noviembre de 2026).

[4] RESAK, “Memorias,” https://resak-jeandavid.surge.sh/memorias.html (consulta: 8 de noviembre de 2026).

[5] Oracle, “MySQL 8.0 Reference Manual: CHECK Constraints,” https://dev.mysql.com/doc/refman/8.0/en/create-table-check-constraints.html

[6] Oracle, “MySQL 8.0 Reference Manual: FOREIGN KEY Constraints,” https://dev.mysql.com/doc/refman/8.0/en/create-table-foreign-keys.html

ANEXOS

- Diagram:https://drive.google.com/file/d/1sJBCLi6HMAOiFtYepOWm_eGQT7LQ0lk0/view?usp=sharing
- Github: https://github.com/LucasOMSExodos/Proyecto-Base-de-Datos